JWT 게시판 구현 보고서

01 서비스 소개와 구현 범위

학번 **202212953** 이름: 성윤오 제출일: 2026.10.28

서비스 이름 우리의 학습 기록

기술 조합과 버전 React / Python FastAPI / SQLite / Windows

GitHub 저장소 URL **https://github.com/s44158619-code/JWT-Assignment**

소스 코드 기준 main 브랜치 / cb9573e

서비스의 대상 사용자와 사용 목적을 3~5 문장으로 설명하세요. 이 기술 조합을 선택한 이유와 제공 템플릿에서 변경한 범위를 적으세요.

본 서비스 이름은 '우리의 학습 기록'이며, 프로그래밍과 웹 개발 등 다양한 학습 내용을 기록하고 공유하기 위한 게시판이다. 누구나 게시글 목록과 상세 내용을 조회할 수 있으며, 로그인한 사용자는 자신의 게시글을 작성하고 수정·삭제할 수 있다. 프론트엔드는 사용자 상태와 화면을 관리하기 편리한 React 를 선택하였고, 백엔드는 Python 기반으로 REST API 를 구현할 수 있는 fastAPI 를 선택하였다. 데이터 저장에는 별도의 서버 설치가 필요 없는 SQLite 를 사용하였다.

교수님이 제공한 시작 템플릿의 게시판 CRUD, 데이터 저장, 테스트 계정 기능을 활용하고, 미완성으로 제공된 JWT 발급·검증, 작성자 권한 검사, React 로그인 연결 및 오류 처리 기능을 구현하였다.

기능	완료 여부	관련 파일	비고 또는 제한
공개 목록과 상세	[완료]	[app.py, store.py, main.jsx]	[비로그인 조회 가능]
로그인과 JWT 발급	[완료]	[auth.py, app.py]	[HS256, 기본 TTL 90 초]
인증과 내 정보 조회	[완료]	[auth.py, app.py]	[JWT 검증 후 사용자 반환]
본인 글 작성·수정·삭제	[완료]	[auth.py, app.py, store.py]	[타인 변경 403 거부]
만료·로그아웃·오류 표시	[완료]	[api.js, main.jsx]	[401 시 로그인 상태 해제]
DB 데이터 유지	[완료]	[store.py]	[SQLite, 서버 재시작 후 유지]

구조와 변경 내역

본 웹서비스는 React 프론트엔드, FastAPI 백엔드, SQLite 데이터베이스로 구성된다.
사용자 브라우저(React) -> Vite 프록시 -> FastAPI 서버 -> SQLite 데이터베이스 순서로 요청이 처리된다.
프론트엔드는 HTTP 요청을 전송하고, 백엔드는 입력값과 인증 정보를 검증한 후 데이터베이스 작업 결과를 JSON 으로 반환한다.

로그인 요청 흐름: 사용자가 이메일과 비밀번호를 입력하면 React 가 POST/api/auth/login 요청을 보낸다. FastAPI 는 제공된 계정과 비밀번호를 검사한 후 JWT 를 발급한다. React 는 JWT 와 사용자 정보를 브라우저 메모리에 저장하고 로그인 상태를 표시한다.

게시글 수정 요청 흐름: 로그인 사용자가 수정 버튼을 누르면 React 가 Authorization: Bearer 헤더에 JWT 를 포함하여 PUT /api/posts/{id} 요청을 보낸다. FastAPI 는 토큰을 검증하고 해당 게시글의 작성자 ID 와 현재 사용자 ID 를 비교한다. 작성자가 같으면 수정하고, 다르면 403 FORBIDDEN 을 반환한다.

JWT 는 브라우저 메모리에만 저장하며 localStorage 또는 sessionStorage 에는 저장하지 않는다. 공개 게시글 조회에는 토큰이 필요하지 않다.

변경한 파일과 함수	제공 코드의 원래 역할	내가 변경한 내용	변경 이유
auth.py / issue_token()	JWT 발급 TODO, 501 반환	HS256 JWT 발급 구현	로그인 인증 완성
auth.py / current_user_id()	데모 사용자 ID 반환	JWT 서명·만료·클레임 검증	실제 사용자 인증
auth.py / assert_owner()	작성자 검사 미구현	사용자 ID 와 작성자 ID 비교	타인 글 변경 방지
api.js / login()	로그인 연결 미완성	로그인 API 호출 및 토큰 저장	화면과 JWT 연결
api.js / request()	API 요청 처리	토큰 첨부 및 401 오류 처리	보호 API 인증
main.jsx / App()	기본 게시판 화면	로그인 상태·권한별 버튼·오류 안내	사용자 화면 완성

게시판 CRUD, SQLite 저장 및 Alice·Bob 테스트 계정은 교수님이 제공한 시작 코드를 활용하였다. JWT 발급·검증과 작성자 권한 검사는 ChatGPT 가 제안한 코드를 바탕으로 직접 입력하여 구현하였다. React 의 API 요청 처리와 로그인 화면도 AI 제안 코드를 반영하였으며, 실제 VS Code 에서 실행하고 PowerShell 및 브라우저를 이용해 테스트하였다.

로컬 실행 기록

실제 설치·빌드·실행 명령과 결과를 적고, 자세한 재현 방법은 저장소 README 에서 연결하세요.

[운영체제와 버전 / 새 폴더 설치 / 환경설정 / 프론트 빌드 / 백엔드 실행 / SQLite 파일 경로]

[4173 번 접속 및 API 연결 확인 / 서버 재시작 후 데이터 유지 / 증거 위치]

운영체제와 버전 : Microsoft Windows 11 Home(버전 10.0.26200) 환경에서 Vs Code 와 PowerShell 을 이용하여 구현함.

프론트 빌드 : React 19.3.0, Vite 8.3.4 를 사용
백엔드 실행 : Python 3.10.11 기반의 FastAPI 0.143.0, Uvicorn 0.54.0 을 사용
JWT 인증 : PyJWT 를 사용
데이터베이스 : SQLite 를 사용
NOde.js : 24.11.0 / npm : 11.6.1 / Git : 2.51.0.windows.1 사용

프론트엔드는 npm ci 로 의존성을 설치한 후 npm run build 명령으로 포로덕션 빌드를 수행하고, npm run build 명령으로 프로덕션 빌드를 수행하고, npm run preview 명령으로 빌드된 화면을 실행하였다.
브라우저에서 http://127.0.0.1:4173 에 접속하여 게시글 조회·등록·수정·삭제 및 로그인 기능을 확인하였다.

SQLite 데이터베이스는 backend-fastapi/data/board.sqlite 경로를 사용하였다. 게시글 작성 후 FastAPI 서버를 종료하고 재시작하여 게시글 제목, 내용, 작성자 정보가 유지되는지 확인하였다.

React 빌드 성공 조건은 docs/evidence/05_build.png, SQLite 데이터 유지 증거는 docs/evidence/11_persistence.png 에 저장하였다.

백엔드는 Python 가상환경(.venv)을 생성한 후 requirements.txt 를 이용하여 필요한 라이브러리를 설치하였다. JWT_SECRET 은 64 자리 16 진수 난수로 생성하여 .env 파일에 설정하였다. 이후 python -m uvicorn app:app --host 127.0.0.1 -- port 8000 명령으로 FastAPI 서버를 실행하였다.

핵심 코드 설명

3 1 로그인과 JWT 발급

파일 / 함수: backend-fastapi/auth.py / issue_token()

코드 발췌

[핵심 코드 5~20 줄을 붙여 넣으세요. 필요한 만큼 영역을 늘립니다.]

```
def issue_token(user):
    secret = jwt_config()
    ttl = int(os.getenv("TOKEN_TTL", "90"))
    now = datetime.now(timezone.utc)
    payload = {
        "sub": str(user["id"]),
        "exp": now + timedelta(seconds=ttl),
        "iss": os.getenv("JWT_ISSUER", "webservice-class"),
        "aud": os.getenv("JWT_AUDIENCE", "board-api"),
    }
    return jwt.encode(payload, secret, algorithm="HS256")
```

동작과 응답 코드 설명

[입력 검증, 계정 확인, 토큰 클레임과 유효 시간, 응답을 설명] 성공·실패 HTTP 와 JSON code 도 적으세요.]
입력값은 로그인에 성공한 사용자 정보이다. 서버는 환경변수의 JWT_SECRET 을 사용하여 HS256 알고리즘으로 JWT 를 발급한다. 토큰에는 사용자 ID 인 sub, 만료 시각 exp, 발급자 iss, 사용 대상 aud 를 포함한다. 기본 유효 시간은 90 초이며, 테스트에서는 TOKEN_TTL 을 변경할 수 있다. 비밀번호와 해시는 토큰에 포함하지 않는다.

로그인 성공 시 HTTP 200 과 accessToken, expiresIn, user 를 반환한다. 존재하지 않는 이메일이나 잘못된 비밀번호는 HTTP 401 / INVALID_CREDENTIALS 로 처리하고, 입력 형식이 올바르지 않은 경우에는 HTTP 400 / BAD_INPUT 을 반환한다. 로그인 응답의 분기는 app.py 의 로그인 처리 코드와 연결된다.

3 2 브라우저 토큰 저장과 전송

파일 / 함수: [frontend-react/src/api.js / login(), authorizationHeaders()

코드 발췌

[핵심 코드 5~20 줄을 붙여 넣으세요. 필요한 만큼 영역을 늘립니다.]

```
let session = { accessToken: "", user: null };
function authorizationHeaders() {
  if (!session.accessToken) {
    return {};
```

```
  }
  return {
    Authorization: `Bearer ${session.accessToken}`,
  };
}
export function clearSession() {
  session = { accessToken: "", user: null };
}
```

동작과 응답 코드 설명

[로그인 응답 처리, 저장 위치, Authorization 헤더, 로그아웃을 설명] 성공·실패 HTTP 와 JSON code 도 적으세요.]
로그인 폼에서 입력한 email 과 password 를 POST /api/auth/login 으로 전송한다. 로그인 성공 시 반환된 JWT 와 사용자 정보를 JavaScript 메모리의 session 객체에 저장한다.

게시글 등록·수정·삭제와 내 정보 조회 요청에는 Authorization: Bearer 형식으로 JWT 를 첨부한다. 토큰이 없는 경우에는 Authorization 헤더를 생성하지 않는다.

성공 응답은 HTTP 200 또는 201 이며, 삭제 성공은 HTTP 204 이다. 인증 오류인 HTTP 401 이 발생하면 메모리의 인증 상태를 초기화한다. 만료 토큰은 TOKEN_EXPIRED, 잘못된 로그인은 INVALID_CREDENTIALS 등에 맞춰 안내한다. HTTP 403 은 권한 부족이므로 자동 로그아웃하지 않는다.

로그아웃 시 clearSession()으로 메모리의 JWT 와 사용자 정보를 제거한다.

서버 인증과 작성자 권한

아래 설명은 실제 제출한 파일과 함수의 동작을 근거로 작성하세요.

3 3 서버 JWT 검증

파일 / 함수: backend-fastapi/auth.py / current_user_id()

코드 발췌

```
[핵심 코드 5~20 줄을 붙여 넣으세요. 필요한 만큼 영역을 늘립니다.]
try:
    payload = jwt.decode(
        token,
        jwt_config(),
        algorithms=["HS256"],
        issuer=os.getenv("JWT_ISSUER", "webservice-class"),
        audience=os.getenv("JWT_AUDIENCE", "board-api"),
        options={
            "require": ["sub", "exp", "iss", "aud"]
        },
    )
except jwt.ExpiredSignatureError:
    raise ApiError(401, "TOKEN_EXPIRED", "토큰이 만료되었습니다.")
except jwt.InvalidTokenError:
    raise ApiError(401, "INVALID_TOKEN", "유효하지 않은 토큰입니다.")
```

동작과 응답 코드 설명

[서명·알고리즘·만료·필수 클레임 검사와 사용자 ID 전달, 실패 응답을 설명] 성공·실패 HTTP 와 JSON code 도 적으세요.]
서버는 보호 API 요청의 Authorization 헤더에서 Bearer 토큰을 추출한다. 헤더가 없거나 형식이 잘못된 경우 HTTP 401 / TOKEN_REQUIRED 를 반환한다.

토큰이 존재하면 PyJWT 라이브러리로 HS256 서명, 만료 시간, 발급자, 사용 대상 및 필수 클레임을 검증한다. 서명이나 클레임이 잘못된 토큰은 HTTP 401 / INVALID_TOKEN 으로 거부하고, 정상적으로 서명되었지만 만료된 토큰은 HTTP 401 / TOKEN_EXPIRED 로 처리한다.

검증이 성공하면 sub 를 사용자 ID 로 해석하고 실제 사용자 계정이 존재하는지 확인한다. 검증된 사용자 ID 만 게시글 등록·수정·삭제 및 내 정보 조회에 전달한다. 이 부분이 요청 주체 연결점이다.

검증에 성공한 /api/me 요청은 HTTP 200 으로 사용자의 id, name, email 만 반환하며, 비밀번호나 비밀번호 해시는 반환하지 않는다.

3 4 작성자 권한 검사

파일 / 함수: backend-fastapi/auth.py / assert_owner()

코드 발췌

```
[핵심 코드 5~20 줄을 붙여 넣으세요. 필요한 만큼 영역을 늘립니다.]
def assert_owner(user_id, author_id):
    if user_id != author_id:
        raise ApiError(
            403,
            "FORBIDDEN",
            "본인이 작성한 게시글만 수정하거나 삭제할 수 있습니다."
        )
```

동작과 응답 코드 설명

[누구와 누구를 비교하는지, 401 과 403 의 차이, 작성자 위조 방지를 설명] 성공·실패 HTTP 와 JSON code 도적으세요.]

게시글의 수정·삭제 요청을 처리할 때 서버는 JWT 검증으로 확인한 사용자 ID 와 SQLite 에 저장된 게시글의 authorId 를 비교한다. 이 비교 과정이 권한 경계 대조에 해당한다.

두 ID 가 일치하면 수정 또는 삭제를 진행하고, 일치하지 않으면 HTTP 403 / FORBIDDEN 을 반환하며 데이터베이스를 변경하지 않는다. 정상 수정은 HTTP 200, 정상 삭제는 HTTP 204 를 반환한다.

HTTP 401 은 토큰이 없거나 유효하지 않아 인증에 실패한 경우이며, HTTP 403 은 정상적으로 인증된 사용자가 다른 사람의 게시글을 변경하려는 경우이다.

글을 등록할 때도 클라이언트가 보낸 authorId 를 신뢰하지 않고, 서버가 JWT 로 확인한 사용자 ID 를 작성자로 지정한다. 요청 본문에 authorId 또는 userId 등 허용되지 않은 필드를 추가하면 HTTP 400 / BAD_INPUT 으로 처리하여 작성자 위조를 방지한다.

테스트 결과

T01 부터 T17 까지 수행하고, 실제 응답과 증거의 위치를 기록하세요.

실행 환경:
- 운영체제: Microsoft Windows 11 Home (10.0.26200)

- 프론트엔드: React 19.3.0 / Vite 8.3.4

- 백엔드: Python 3.10.11 / FastAPI 0.143.0 / Uvicorn 0.54.0

- JWT 라이브러리: PyJWT 2.15.1

- Node.js: 24.11.0 / npm: 11.6.1

- 데이터베이스: SQLite (data/board.sqlite)

- 브라우저: Google Chrome

- 테스트 도구: Windows PowerShell, Chrome 개발자 도구 Network, 교수님 제공 check_api.py

로컬 배포 확인:
백엔드는 python -m uvicorn app:app --host 127.0.0.1 --port 8000 명령으로 실행하였다. 프론트엔드는 npm ci, npm run build, npm run preview 순서로 실행하였다. 브라우저에서 http://127.0.0.1:4173 에접속하여 로그인과 게시글 CRUD 기능 및 FastAPI 연동을 확인하였다.

SQLite 데이터 유지 검사를 위해 게시글 작성 후 백엔드 서버를 실제로 종료·재시작하고, 기존 게시글의 제목·내용·작성자가 유지되는지 확인하였다.

새 폴더에서 GitHub 저장소를 clone 하여 재현하는 검사는 최종 GitHub 업로드 후 수행할 예정이다.

테스트 일시 및 코드 버전:

테스트 일시: [2026.10.28]

코드 버전: 로컬 구현본 (GitHub 업로드 후 기준 커밋 SHA 입력)

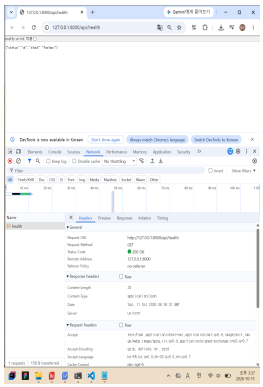
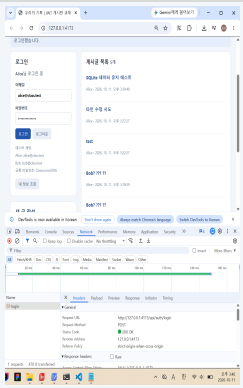
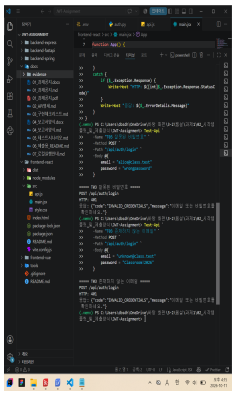
일반 테스트 JWT TTL: 90 초

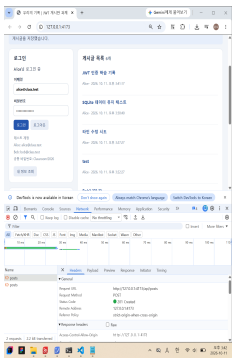
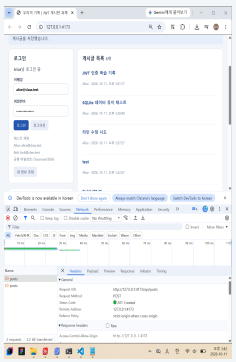
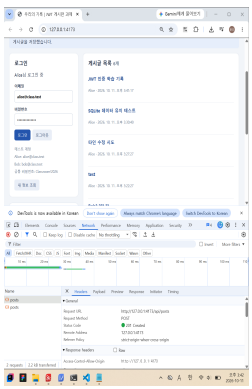
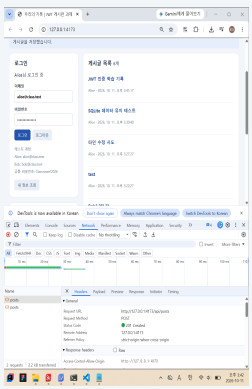
만료 테스트 JWT TTL: 3 초

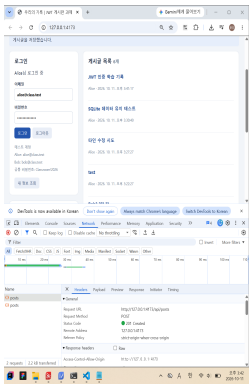
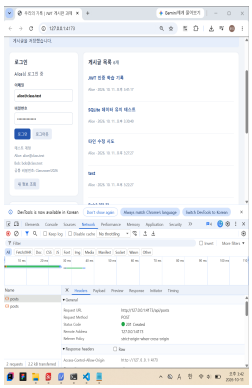
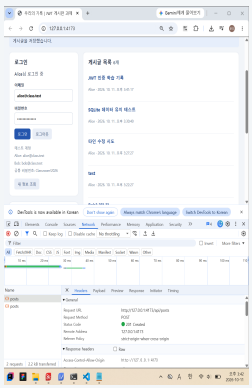
토큰 TTL: [일반 테스트 값 / 만료 테스트 값]

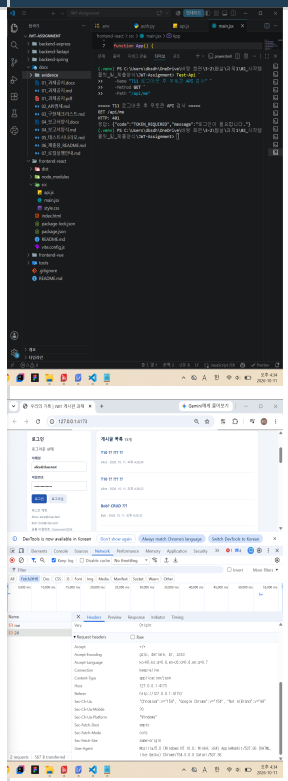
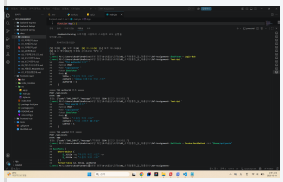
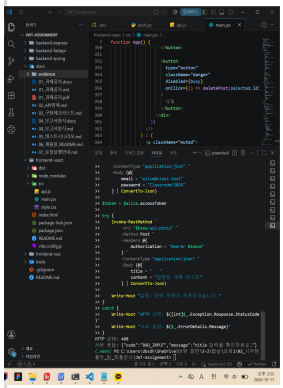
05_테스트시나리오.md 의 T01~T17 결과를 모두 기록하세요. PASS, FAIL, 미실행을 구분합니다. 예상 결과를 실제 결과로 복사하지 말고 직접 관찰한 HTTP 상태와 JSON code, 화면을 적으세요. 실제 결과 칸에는 예를 들어 403 / FORBIDDEN 처럼 기록합니다. 정상 응답은 200 / 해당 없음처럼 적습니다.

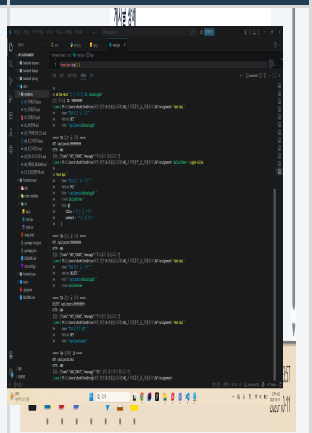
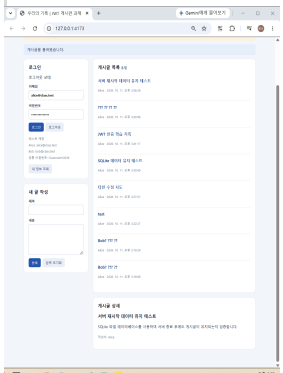
ID	수행 절차와 사용자	예상 결과	실제 HTTP 와 code	판정	증거

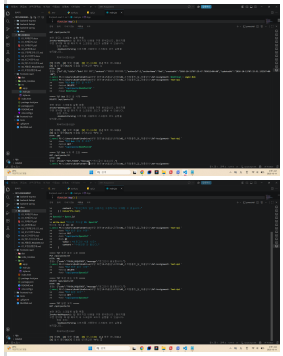
ID	수행 절차와 사용자	예상 결과	실제 HTTP 와 code	판정	증거
T01	비로그인 상태에서 게시글 목록 및 상세 조회	200	200 / 해당 없음	PASS	
T02	Alice 계정으로 정상 로그인	200, JWT 발급	200 / 해당 없음	PASS	
T03	잘못된 비밀번호와 존재하지 않는 이메일로 로그인	401 INVALID_CREDENTIALS	각각 401 / INVALID_CREDENTIALS	PASS	
T04	JWT 없이 /api/me 와 글 등록 요청	401 TOKEN_REQUIRED	각각 401 / TOKEN_REQUIRED	PASS	

ID	수행 절차와 사용자	예상 결과	실제 HTTP 와 code	판정	증거
T05	Alice JWT 로 글 등록 및 /api/me 조회	201, 200	글 등록 201 / 해당 없음, 내 정보 200 / 해당 없음	PASS	
T06	Alice JWT 로 본인 글 수정·삭제	200, 204	수정 200 / 해당 없음, 삭제 204 / 본문 없음	PASS	
T07	Bob JWT 로 Alice 글 수정 시도	403 FOBBIDDEN	403 / FORBIDDEN, 원본 유지	PASS	
T08	Bob JWT 로 Alice 글 삭제 시도	403 FOBBIDDEN	403 / FORBIDDEN, 게시글 유지	PASS	

ID	수행 절차와 사용자	예상 결과	실제 HTTP 와 code	판정	증거
T09	서명 변조 JWT 로 /me 및 글 등록·수정·삭제	401 INVALID_TOKEN	각각 401 / INVALID_TOKEN	PASS	 
T10	TTL 3 초로 발급한 JWT 만료 후 보호 API 요청	401 TOKEN_EXPIRED	401 / TOKEN_EXPIRED, React 로그인 해제	PASS	

ID	수행 절차와 사용자	예상 결과	실제 HTTP 와 code	판정	증거
T11	Alice 로그인 후 로그아웃, 토큰 없는 /me 요청	401 TOKEN_REQUIRED	401 / TOKEN_REQUIRED, 로그아웃 상태	PASS	
T12	Bob 이 authorId:1, userId:1 추가하여 글 등록	400 BAD_INPUT	각각 400 / BAD_INPUT, 위조 글 미생성	PASS	
T13	빈 제목·공백 본문·길이 초과 입력	400 BAD_INPUT	각각 400 / BAD_INPUT, 글 미생성	PASS	 

ID	수행 절차와 사용자	예상 결과	실제 HTTP 와 code	판정	증거
T14	없는 ID 조회·수정·삭제 및 잘못된 ID 조회	404, 400	없는 글 404 / NOT_FOUND, 잘못된 ID 400 / BAD_INPUT	PASS	
T15	글 작성 후 FastAPI 서버 종료·재시작	데이터 유지	재조회 200 / 해당 없음, 글 유지	PASS	 
T16	로그인 후 페이지 새로고침	로그인 해제, 공개 글 유지	공개 조회 정상, 로그인 상태 해제	PASS	

ID	수행 절차와 사용자	예상 결과	실제 HTTP 와 code	판정	증거
T17	토큰 없는 수정·삭제, Bob 자신의 글 CRUD	401, 정상 CRUD	무토큰 401 / TOKEN_REQUIRED, Bob 수정 200·삭제 204	PASS	

HTTP 응답 확인 기록

예상 코드와 실제 코드를 대조하고, 201·204·401 의 헤더와 본문도 확인하세요.

숫자는 Network 나 API 클라이언트에서 직접 관찰한 값을 기록합니다. 본문에 있는 숫자만으로 판단하지 않습니다. 한 항목이 여러 요청을 포함하면 행을 추가하세요.

확인 항목	기대 응답	실제 HTTP 와 code	증거 위치
정상 조회·로그인·수정	**200**	200 / 해당 없음	02_login.png, T06_update.png
새 글 등록	**201 및 Location**	201 / 해당 없음, Location: /api/posts/25	29_201_location.png
본인 글 삭제	**204 및 빈 본문**	204 / 본문 없음 (길이 0)	30_204_empty.png
입력 오류·추가 작성자 필드	**400 BAD_INPUT**	400 / BAD_INPUT	10_bad_input.png, 19_T12_author_spoof.png
잘못된 계정	**401 INVALID_CREDENTIALS**	401 / INVALID_CREDENTIALS	15_T03_invalid_login.png
토큰 없음·헤더 형식 오류	**401 TOKEN_REQUIRED**	401 / TOKEN_REQUIRED, 형식 오류 [확인]	27_401_header.png, 27_401_bad_format.png
변조·클레임 오류	**401 INVALID_TOKEN**	401 / INVALID_TOKEN(서명 변조 및 aud 불일치)	07_invalid_token.png, 31_invalid_audience.png
정상 서명의 만료 토큰	**401 TOKEN_EXPIRED**	401 / TOKEN_EXPIRED	08_expired_token.png
다른 작성자의 변경	**403 FORBIDDEN**	403 / FORBIDDEN	04_permission.png , 17_T08_forbidden_delete.png

확인 항목	기대 응답	실제 HTTP 와 code	증거 위치
없는 글	**404 NOT_FOUND**	404 / NOT_FOUND	21_T14_not_found.png

[보호 API 401 의 WWW-Authenticate: Bearer 확인 결과와 증거]

JWT 없이 GET /api/me 요청을 전송한 결과, 서버가 HTTP 401 Unauthorized 와 JSON code TOKEN_REQUIRED 를 반환하는 것을 확인하였다. 또한 응답 헤더에서 WWW-Authenticate: Bearer 값을 확인하여 인증이 필요한 API 에서 올바른 인증 실패 응답을 반환함을 검증하였다.

증거: docs/evidence/27_401_header.png

추가로 Authorization 헤더에 정상적인 Bearer 형식 대신 Basic abc 를 전달하여 요청하였다. 서버는 잘못된 인증 헤더 형식을 거부하고 HTTP 401 / TOKEN_REQUIRED 를 반환하였으며, WWW-Authenticate: Bearer 응답 헤더도 포함하는 것을 확인하였다.

증거: docs/evidence/27_401_bad_format.png

[401 과 403 에서 화면이 각각 어떻게 달라지는지 설명]

HTTP 401 은 JWT 가 없거나 유효하지 않은 경우의 인증 실패 응답이다. React 는 보호 API 요청에서 401 을 받으면 메모리에 저장된 JWT 와 사용자 정보를 제거하여 로그인 상태를 해제한다. 반면 HTTP 403 은 정상적으로 로그인한 사용자가 타인의 게시글을 변경하려는 경우의 권한 부족 응답이므로, 오류를 표시하되 로그인 상태를 유지한다.

[실패한 수정·삭제 뒤 공개 조회로 데이터 불변을 확인한 결과]

Bob 의 JWT 를 사용하여 Alice 가 작성한 게시글에 수정 및 삭제 요청을 보낸 결과, 서버는 HTTP 403 / FORBIDDEN 을 반환하였다. 이후 공개 조회 API 로 해당 게시글을 다시 조회하여 기존 제목과 내용이 변경되지 않았고 게시글이 삭제되지 않았음을 확인하였다.

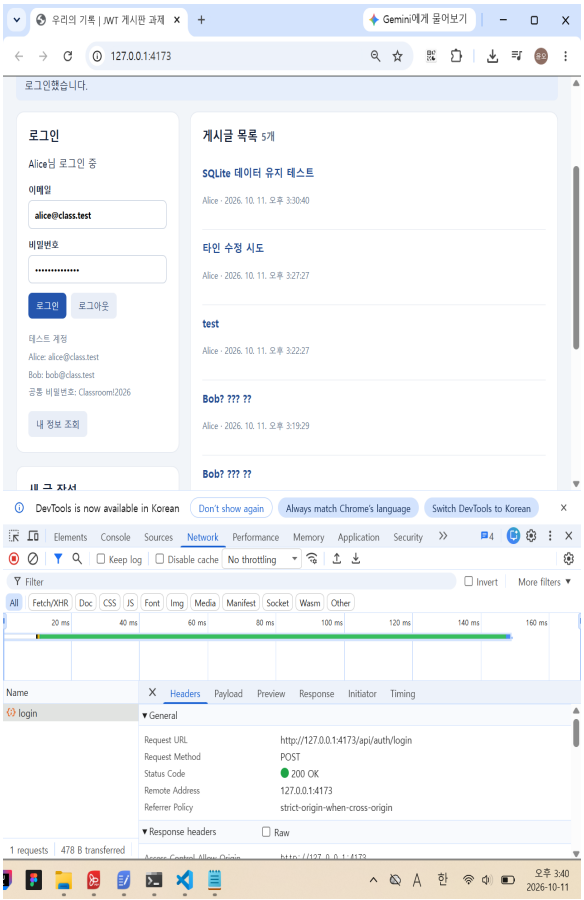
관련 증거: 04_permission.png, 17_T08_forbidden_delete.png

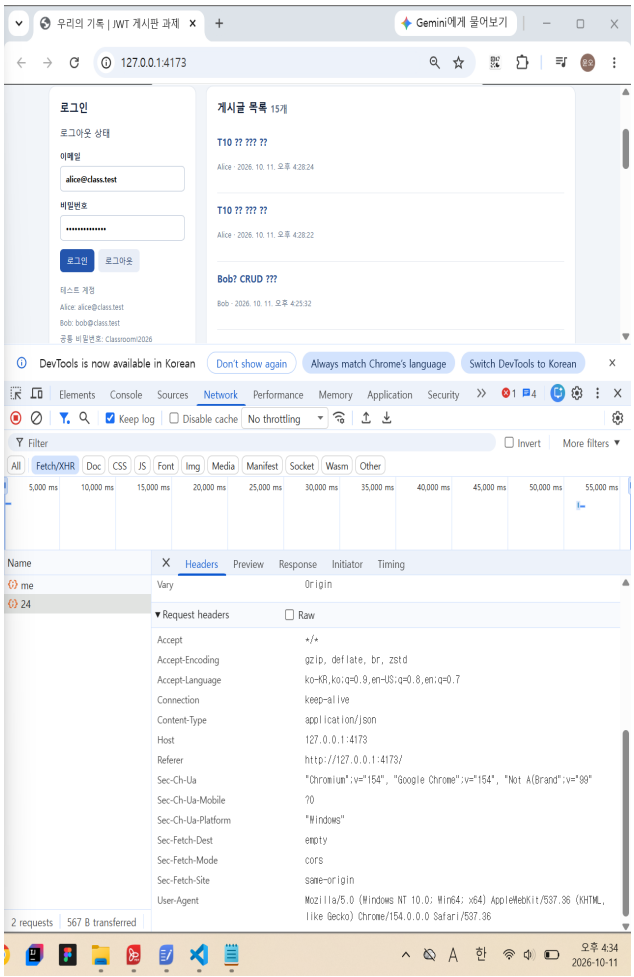
테스트 증거

로그인과 토큰 전송

[증거 1: 로그인 성공과 토큰이 첨부된 요청. 토큰 원문은 가리세요.]

[화면 캡처 또는 요청·응답 삽입]
Alice 계정으로 로그인하여 POST /api/auth/login 요청에서 HTTP 200 응답을 확인하였다. 로그인 성공 후 사용자 이름이 화면에 표시되는지 확인하고, 보호 API 요청에 Authorization: Bearer 헤더가 포함되는지 검사하였다.

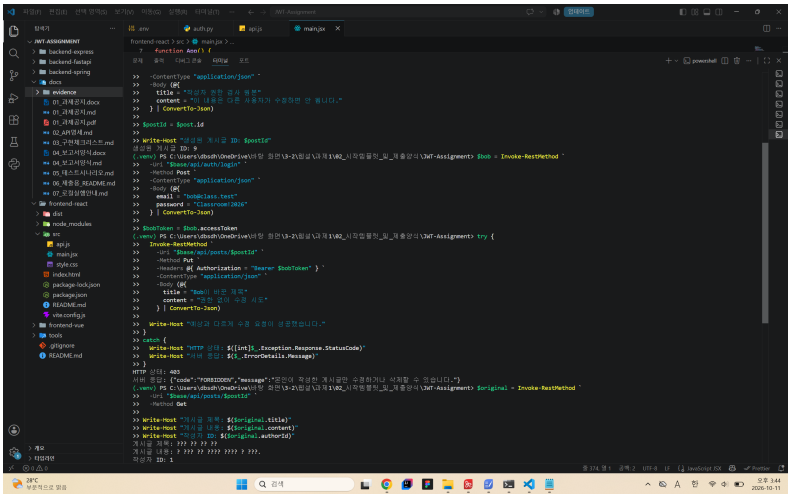




다른 작성자의 수정과 삭제

[증거 2: Bob 이 Alice 의 글 수정·삭제를 시도한 서버 응답]

[화면 캡처 또는 요청·응답 삽입]
Bob 계정의 JWT 를 이용하여 Alice 가 작성한 게시글을 수정·삭제하는 요청을 전송하였다. 서버는 HTTP 403 / FORBIDDEN 으로 요청을 거부하였으며, 이후 공개 조회로 원본 게시글이 유지되는지 검사하였다.

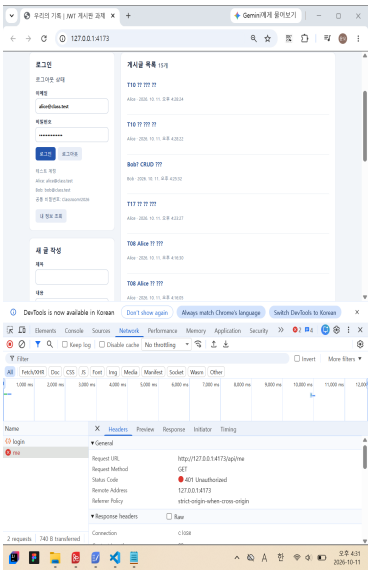


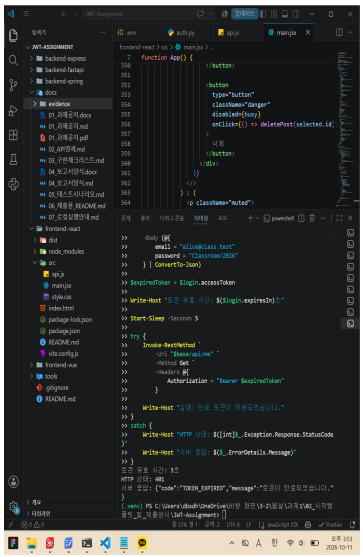


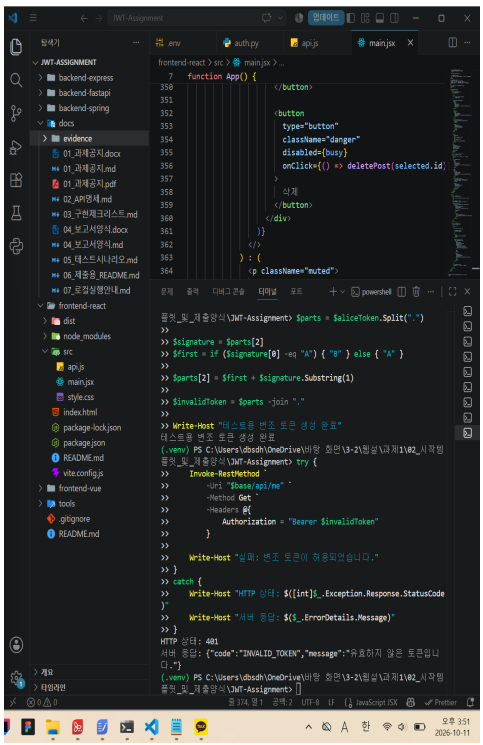
토큰 변조와 만료

[증거 3: 토큰 변조·만료 시 서버 응답과 화면 처리]

[화면 캡처 또는 요청·응답 삽입]
정상 JWT 의 서명을 변경하여 서버에 요청하였고, HTTP 401 / INVALID_TOKEN 응답을 확인하는 테스트를 수행하였다. 또한 TOKEN_TTL 을 3 초로 설정하고 5 초 이상 기다린 후 요청하여 HTTP 401 / TOKEN_EXPIRED 응답과 React 로그인 해제 처리를 검사하였다.



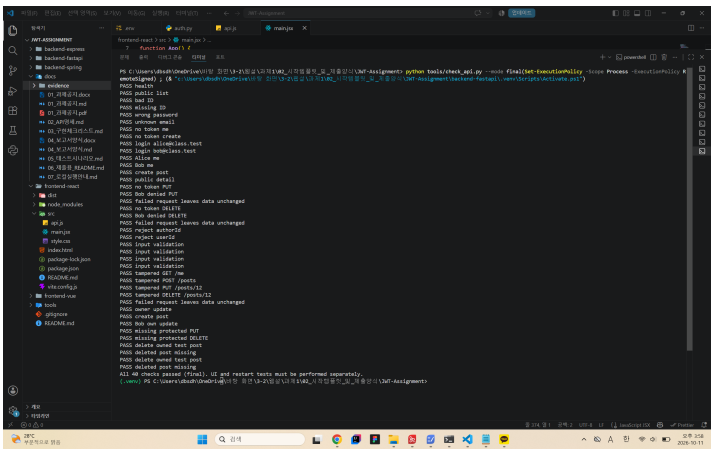


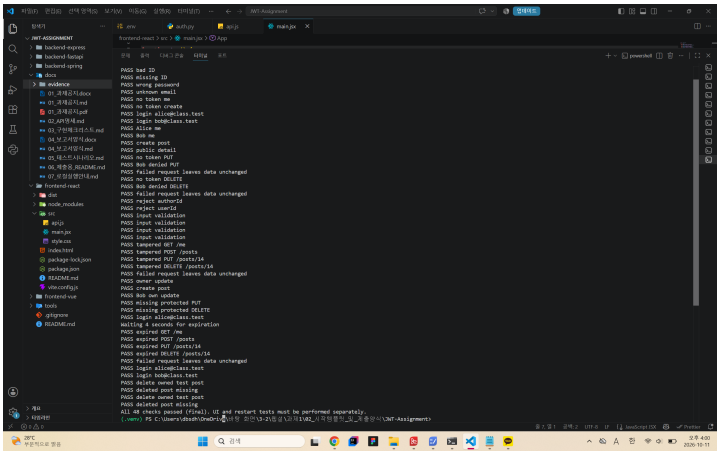


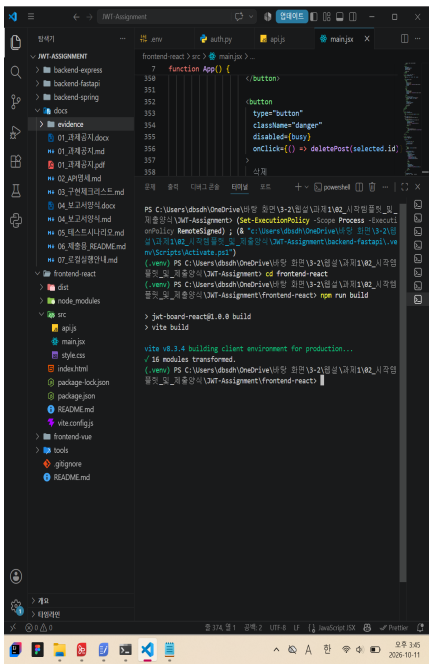
자동 검사와 로컬 배포 증거

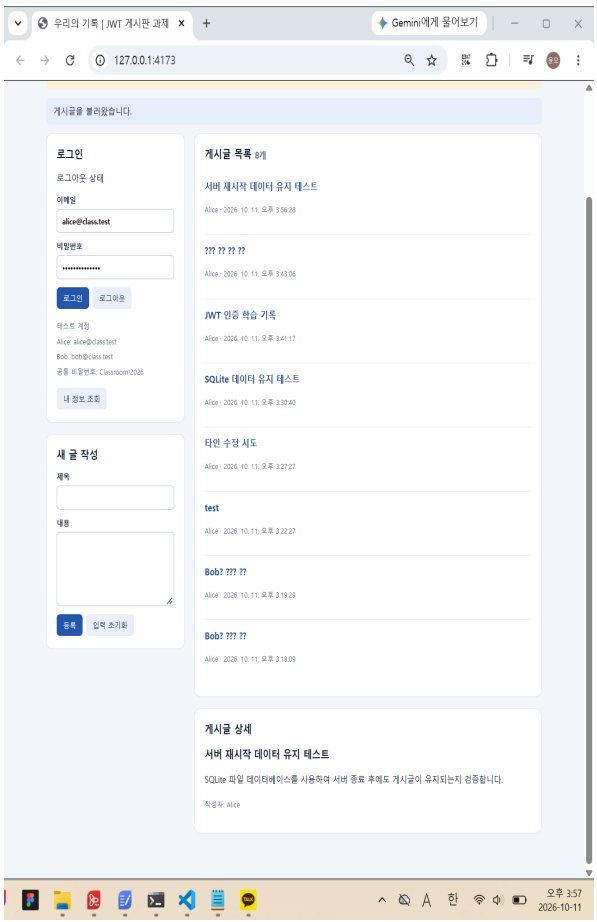
[자동 검사 결과를 첨부했다면 실행 명령과 의미를 설명]

[빌드 성공 로그 / 4173 번 API 연결 / 재시작 유지의 화면 또는 파일 경로]
교수님 제공 자동 검사 도구를 이용하여 JWT 인증과 게시글 CRUD 및 권한 처리를 검사하였다. React 는 npm run build 로 빌드하고 npm run preview 를 통해 4173 번 포트에서 실행하였다. 또한 FastAPI 서버를 재시작하여 SQLite 에 저장된 게시글 데이터가 유지되는지 검사하였다.








우리의 기록 | JWT 게시판 과제
Gemini에게 물어보기
127.0.0.1:4173
게시글을 불러왔습니다.
로그인
로그아웃 상태
이메일
alice@class.test
비밀번호
로그인
로그아웃
테스트 계정
Alice: alice@class.test
Bob: bob@class.test
공통 비밀번호: Classroom2026
내 정보 조회
새 글 작성
제목
내용
등록
입력 초기화
게시글 목록 8개
서버 재시작 데이터 유지 테스트
Alice · 2026. 10. 11. 오후 3:56:28
??? ?? ?? ??
Alice · 2026. 10. 11. 오후 3:43:06
JWT 인증 학습 기록
Alice · 2026. 10. 11. 오후 3:41:17
SQLite 데이터 유지 테스트
Alice · 2026. 10. 11. 오후 3:30:40
타인 수정 시도
Alice · 2026. 10. 11. 오후 3:27:27
test
Alice · 2026. 10. 11. 오후 3:22:27
Bob? ??? ??
Alice · 2026. 10. 11. 오후 3:19:29
Bob? ??? ??
Alice · 2026. 10. 11. 오후 3:18:09
게시글 상세
서버 재시작 데이터 유지 테스트
SQLite 파일 데이터베이스를 사용하여 서버 종료 후에도 게시글이 유지되는지 검증합니다.
작성자: Alice
오후 3:57
2026-10-11

AI 활용과 참고 자료

사용 범위, 실제 반영 위치와 직접 확인한 내용을 구분해 기록하세요.

AI 사용 여부: 사용

사용하지 않았다면 미사용이라고 명시하고 참고한 수업 자료·공식 문서의 출처를 작성하세요. 사용했다면 전체 사용 범위를 요약하고 대표 사례 2~3 건을 작성하세요. 사용이 1 건이면 그 1 건만 적습니다. 대화 전체 제출은 필요하지 않습니다.

도구와 모델	사용 목적	반영한 파일 또는 보고서 항목	반영 방식
ChatGPT / GPT-6	JWT 발급·검증 및 권한 검사 코드 작성	backend-fastapi/auth.py	AI 제안 코드를 직접 입력하고 실행 결과 검증
ChatGPT / GPT-6	React 로그인·로그아웃 및 API 오류 처리	frontend-react/src/api.js, src/main.jsx	AI 초안을 반영하고 브라우저에서 동작 검증
ChatGPT / GPT-6	PowerShell 테스트 및 오류 해결	T01~T17 테스트, 추가 HTTP 검사	AI 가 제안한 명령 실행, 오류 발생 시 수정 후 재검증
ChatGPT / GPT-6	README 및 보고서 작성 보조	README.md, 보고서 01~06	문장 초안 참고 후 실제 코드·테스트 결과와 대조

본 과제에서는 ChatGPT 를 JWT 인증 기능 구현, React 화면과 API 연결, 테스트 명령 작성, 오류 해결, README 및 보고서 문장 정리에 활용하였다.

JWT 발급·검증과 작성자 권한 검사에서는 AI 가 제안한 코드의 구조를 참고하여 VS Code 에서 직접 작성하였다. React 프론트엔드에서는 로그인 응답 처리, 메모리 기반 JWT 저장, Authorization 헤더 첨부 및 HTTP 오류 처리 코드를 반영하였다.

이후 PowerShell 과 브라우저를 이용해 실제 동작을 검사하였으며, 교수님이 제공한 자동 API 검사 도구도 실행하였다. AI 가 제안한 명령어에서 오류가 발생한 경우에는 오류 메시지를 확인한 후 수정하여 다시 실행하였다.

특히 PowerShell 배열 처리 오류와 Python 환경변수 파일 탐색 오류를 해결하는 과정에서 AI 의 디버깅 도움을 받았다. 최종 코드와 보고서 내용은 실제 구현 파일 및 테스트 결과에 맞춰 확인하였다.

대표 사례

대표 사례 1. JWT 발급·검증 및 작성자 권한 검사 구현

도구 / 날짜:ChatGPT / 2026.10.11

요청한 내용:교수님이 제공한 FastAPI 시작 템플릿에서 미완성인 JWT 인증 기능을 구현하는 방법을 질문하였다. HS256 방식으로 JWT 를 발급하고, 보호 API 에서 토큰의 서명과 만료 시간을 검증하며, 다른 작성자의 게시글 수정·삭제를 막는 코드를 요청하였다.

AI 가 제안한 내용:auth.py 에 issue_token(), current_user_id(), assert_owner()함수를 구현하는 방법을 제안하였다. JWT 에 sub, exp, iss, aud 클레임을 포함하고 기본 유효 시간을 90 초로 설정하였다. 서버에서는 JWT 서명·만료·클레임을 검증하고, 게시글 변경 시 인증된 사용자 ID 와 작성자 ID 를 비교하도록 하였다.

실제 반영 위치:backend-fastapi/auth.py 의 jwt_config(), issue_token(), current_user_id(), assert_owner()함수.

내가 확인하고 수정한 내용:AI 가 제안한 코드를 VS Code 에 직접 입력하고 기존 FastAPI 코드와 연결되는지 확인하였다. JWT 비밀 키가 64 자리 16 진수 형식인지 확인하고, 비밀 키를 소스 코드에 직접 작성하지 않고 .env 파일에서 읽도록 하였다. JWT 의 sub 에는 사용자 ID 를 문자열로 저장하고, 인증에 성공한 사용자의 ID 만 작성자 권한 검사에 사용하도록 하였다. AI 제안 코드에 대해 별도의 알고리즘 변경 없이 실제 실행과 테스트를 통해 동작을 확인하였다.

검증 방법과 결과:T02 에서 Alice 계정으로 로그인하여 HTTP 200 및 JWT 발급을 확인하였다. T07·T08 에서는 Bob 계정이 Alice 의 게시글을 수정·삭제하려고 할 때 HTTP 403 / FORBIDDEN 이 반환되는지 검사하였다. T09 에서는 변조된 JWT 가 HTTP 401 / INVALID_TOKEN 으로 거부되는지 확인하였고, T10 에서는 만료된 JWT 에 대해 HTTP 401 / TOKEN_EXPIRED 가 반환되는지 검사하였다. 추가로 aud 가 잘못된 JWT 에 대해 HTTP 401 / INVALID_TOKEN 응답을 확인하였다.

증거:02_login.png, 04_permission.png, 07_invalid_token.png, 08_expired_token.png, 17_T08_forbidden_delete.png, 31_invalid_audience.png

대표 사례 2. React 로그인·로그아웃 및 JWT 메모리 저장 구현

도구 / 날짜:ChatGPT / 2026.10.11

요청한 내용:FastAPI 에서 발급한 JWT 를 React 화면과 연결하는 방법을 질문하였다. Alice 와 Bob 의 로그인 기능, 보호 API 요청 시 JWT 전송, 본인 게시글에만 수정·삭제 버튼 표시, 로그아웃과 토큰 만료 시 인증 상태 초기화 기능의 구현 방법을 요청하였다.

AI 가 제안한 내용:api.js 에 login(), request(), authorizationHeaders(), clearSession()함수를 구현하고, JWT 와 사용자 정보를 JavaScript 메모리의 session 객체에 저장하는 방식을 제안하였다. 보호 API 를 호출할 때 Authorization: Bearer 헤더를 자동으로 첨부하고, 서버에서 HTTP 401 을 반환하면 인증 상태를 초기화하도록 하였다. 또한 main.jsx 에서 로그인 폼, 게시글 목록, 작성·수정·삭제 화면과 사용자별 버튼 표시를 연결하도록 제안하였다.

실제 반영 위치:frontend-react/src/api.js 의 login(), request(), authorizationHeaders(), clearSession(), canEdit()함수 및 frontend-react/src/main.jsx 의 React App 컴포넌트.

내가 확인하고 수정한 내용:AI 가 제안한 코드를 VS Code 에서 직접 입력하여 기존 FastAPI API 와 연결하였다. JWT 를 localStorage 나 sessionStorage 에 저장하지 않고 JavaScript 메모리에서만 관리하는지 확인하였다. Alice 와 Bob 으로 각각 로그인하여 본인이 작성한 글에만 수정·삭제 버튼이 표시되는지 확인하였다. 또한 로그아웃 후 로그인 화면으로 돌아가는지, 만료된 토큰으로 보호 API 를 요청했을 때 인증 상태가 초기화되는지 검사하였다.

검증 방법과 결과:T02 에서 Alice 계정 로그인 성공과 JWT 발급을 확인하였다. T07·T08 에서는 Bob 이 Alice 의 게시글을 수정·삭제하는 요청에 HTTP 403 / FORBIDDEN 이 반환되는 것을 확인하였다. T10 에서는 JWT 만료 후 HTTP 401 / TOKEN_EXPIRED 와 React 로그인 상태 초기화를 검사하였다. T11 에서는 로그아웃 후 JWT 없이 보호 API 를 호출했을 때 HTTP 401 / TOKEN_REQUIRED 가 반환되는 것을 확인하였다. T16 에서는 브라우저 새로고침 후 메모리에 저장된 인증 정보가 사라지고 공개 게시글은 계속 조회할 수 있는지 검사하였다.

증거:02_login.png, 04_permission.png, 12_logout.png, 25_T10_expired_ui.png, 26_T11_logout_network.png

대표 사례 3. PowerShell 및 Python 테스트 오류 해결

도구 / 날짜:ChatGPT / 2026.10.11, 2026.10.28

요청한 내용:JWT 게시판의 API 테스트를 수행하는 과정에서 발생한 PowerShell 및 Python 오류의 원인을 분석하고, 기존 기능을 변경하지 않으면서 테스트를 정상적으로 실행하는 방법을 질문하였다.

AI 가 제안한 내용:첫 번째로 T14 테스트에서 PowerShell 배열을 정수로 변환하는 오류가 발생했을 때, 배열로 묶인 값을 직접 정수로 처리하지 않고 존재하지 않는 게시글 ID 를 단일 숫자 값으로 지정하여 테스트하도록 제안하였다. 두 번째로 JWT 의 aud 클레임 불일치 테스트에서 Python 의 load_dotenv()가 AssertionError 를 발생시키자, .env 파일의 경로를 명시적으로 지정하는 load_dotenv(dotenv_path=".env")방식을 제안하였다.

실제 반영 위치:T14 의 존재하지 않는 게시글 조회·수정·삭제를 검사하는 PowerShell 명령과, JWT 의 aud 클레임을 검증하기 위한 일회성 Python 테스트 명령. 실제 FastAPI 서비스의 인증 코드인 auth.py 나 React 소스 코드는 이 오류 해결 과정에서 수정하지 않았다.

내가 확인하고 수정한 내용:T14 검사 중 PowerShell 에서 System.Object[]형식의 값을 정수로 변환할 수 없다는 오류를 확인하였다. 이후 존재하지 않는 게시글 ID 를 999999999 라는 단일 숫자로 지정하고 요청을 다시 실행하였다. 또한 Python 표준 입력으로 테스트 코드를 실행할 때 load_dotenv()가 .env 의 위치를 자동으로 탐색하지 못하는 문제가 발생하였다. 이를 해결하기 위해 .env 파일의 위치를 명시하도록 테스트 코드를 변경한 뒤 재실행하였다. 두 문제 모두 서비스의 기능 자체가 아니라 테스트 코드의 실행 방식에서 발생한 오류임을 구분하였다.

검증 방법과 결과:T14 에서는 존재하지 않는 게시글에 대한 요청과 잘못된 ID 요청의 HTTP 응답을 다시 검사하였다. 정상 JWT 로 존재하지 않는 게시글에 수정 요청을 보냈을 때 HTTP 404 / NOT_FOUND 를 확인하였으며, 인증 없이 동일한 요청을 보냈을 때는 HTTP 401 / TOKEN_REQUIRED 를 확인하였다. 추가로 aud 가 서버 설정과 다른 JWT 를 생성하여 GET /api/me 에 요청한 결과 HTTP 401 / INVALID_TOKEN 이 반환되는 것을 확인하였다.

증거:21_T14_not_found.png, 28_no_token_missing_post.png, 28_valid_token_missing_post.png, 31_invalid_audience.png

참고 자료

자료 제목과 URL	사용한 위치	반영 내용 및 출처 표시
웹서비스설계 수업 제공 시작 템플릿 (교수자 제공 파일)	app.py, store.py, users.py, frontend-react/	기본 게시판 CRUD, SQLite 저장, Alice·Bob 테스트 계정 활용. 교수자 제공 코드 기반
웹서비스설계 JWT 게시판 과제 요구사항 (교수자 제공 문서)	auth.py, api.js, main.jsx, 보고서 01~03	JWT HS256, 필수 클레임, 90 초 TTL, 인증·권한 처리 및 API 응답 요구사항 참고
웹서비스설계 05_테스트시나리오.md(교수자 제공 문서)	docs/test-results.md, 보고서 04	T01~T17 테스트 조건, 예상 HTTP 상태와 JSON code, 검증 항목 참고
ChatGPT (OpenAI), https://chatgpt.com/	auth.py, api.js, main.jsx, README, 보고서 05	JWT 구현·React 연동 초안, 테스트 명령 및 오류 해결, 보고서 문장 정리에 활용. 실제 실행 결과와 대조

본 과제에서는 교수님이 제공한 시작 템플릿을 기본 구현 자료로 사용하였다. 게시판의 기본 CRUD 및 SQLite 저장 구조를 유지하면서 과제 요구사항에 따라 JWT 인증, 작성자 권한 검사, React 로그인·로그아웃 기능을 구현하였다.

테스트는 교수님이 제공 테스트 시나리오의 T01~T17 을 기준으로 수행하였으며, 자동 검사 도구와 PowerShell 을 이용하여 실제 HTTP 응답을 확인하였다.

ChatGPT 는 코드 구현 방법 학습, 일부 코드 초안 작성, 테스트 명령 작성, 오류 해결 및 보고서 문장 정리에 활용하였다. AI 가 제안한 내용은 실제 프로젝트 코드와 테스트 결과를 확인한 후 반영하였다.

웹서비스설계 06

이해도 확인과 회고

각 질문에 본인 코드의 동작을 근거로 답하세요.

1. 요청 본문의 authorId 를 믿으면 어떤 문제가 생기며, 내 코드는 어떻게 막는가?

클라이언트가 전송한 요청 본문의 authorId 를 그대로 신뢰하면 Bob 이 Alice 의 사용자 ID 를 입력하여 Alice 가 작성한 것처럼 게시글을 등록하는 작성자 위조가 가능해진다.

이를 방지하기 위해 내 코드는 auth.py 의 current_user_id()에서 JWT 의 서명과 클레임을 검증하고, sub 에 저장된 사용자 ID 를 확인한다. 이후 게시글 등록 API 에서는 요청 본문에 있는 작성자 정보를 사용하지 않고, 서버가 JWT 에서 검증한 사용자 ID 를 작성자로 지정한다.

또한 클라이언트가 authorId 또는 userId 같은 허용되지 않은 필드를 요청 본문에 추가하면 HTTP 400 / BAD_INPUT 으로 거부한다. 이를 통해 인증된 사용자만 자신의 이름으로 게시글을 등록할 수 있도록 하였다.

T12 테스트에서 작성자 ID 를 위조한 요청이 HTTP 400 / BAD_INPUT 으로 거부되는지 확인하였다.

관련 코드: backend-fastapi/auth.py / current_user_id(), backend-fastapi/app.py

관련 테스트: T12

증거: docs/evidence/19_T12_author_spoof.png

2. 수정 버튼을 숨기는 것만으로 충분하지 않은 이유는 무엇이며, 서버의 어떤 함수가 권한을 검사하는가?

React 화면에서 다른 사람이 작성한 게시글의 수정·삭제 버튼을 숨기더라도, 사용자가 PowerShell 이나 API 클라이언트를 통해 서버에 PUT 또는 DELETE 요청을 직접 보낼 수 있으므로 버튼 숨김만으로는 권한을 보호할 수 없다.

내 코드는 프론트엔드의 api.js 에 있는 canEdit() 함수로 본인 글에만 수정·삭제 버튼이 표시되도록 처리하고, 실제 권한 검사는 백엔드 auth.py 의 assert_owner() 함수에서 수행한다.

서버는 JWT 를 검증하여 확인한 사용자 ID 와 SQLite 에 저장된 게시글 작성자 ID 를 비교한다. 두 ID 가 일치하면 수정·삭제를 허용하고, 일치하지 않으면 HTTP 403 / FORBIDDEN 을 반환하여 데이터 변경을 차단한다.

T07 과 T08 테스트에서 Bob 의 JWT 로 Alice 의 게시글을 수정·삭제하려고 했을 때 두 요청 모두 HTTP 403 / FORBIDDEN 으로 거부되는 것을 확인하였다. 따라서 실제 보안은 화면의 버튼 표시가 아니라 서버의 작성자 권한 검사로 보장된다.

관련 코드:

- frontend-react/src/api.js / canEdit()

- backend-fastapi/auth.py / assert_owner()

관련 테스트: T07, T08

증거:

- docs/evidence/04_permission.png

- docs/evidence/17_T08_forbidden_delete.png

3. JWT decode 와 verify 의 차이는 무엇이며, 내 코드는 어떤 검증을 수행하는가?

JWT 의 decode 는 토큰에 포함된 Header 와 Payload 의 내용을 해석하는 것을 의미하며, 단순히 내용을 읽는 것만으로는 토큰이 정상적으로 발급되었는지 신뢰할 수 없다. 반면 verify 는 서명의 유효성과 토큰의 만료 여부, 발급자 및 사용 대상 등의 조건을 확인하여 토큰을 신뢰할 수 있는지 판단하는 과정이다.

내 코드는 backend-fastapi/auth.py 의 current_user_id() 함수에서 PyJWT 의 jwt.decode()를 사용한다. PyJWT 의 jwt.decode()는 기본적으로 서명 검증을 수행하므로 단순한 내용 해석이 아니라 실제 인증 검증에 해당한다.

구체적으로 JWT_SECRET 을 이용한 서명 검증, HS256 알고리즘 제한, exp 만료 시간 확인, iss 발급자 확인, aud 사용 대상 확인을 수행한다. 또한 sub, exp, iss, aud 를 필수 클레임으로 지정하여 누락 여부를 검사한다. 검증에 성공하면 sub 를 사용자 ID 로 해석하고 실제 사용자 계정이 존재하는지도 확인한다.

서명이나 클레임이 유효하지 않으면 HTTP 401 / INVALID_TOKEN 을 반환하며, 만료된 토큰은 HTTP 401 / TOKEN_EXPIRED 로 구분하여 처리한다. 검증에 성공한 사용자 ID 는 이후 보호 API 에서 인증된 요청 주체를 판단하는 데 사용된다.

T09 에서 변조된 JWT 에 대해 401 / INVALID_TOKEN 을 확인하였고, T10 에서 만료된 JWT 에 대해 401 / TOKEN_EXPIRED 를 확인하였다. 추가 검사에서는 aud 값이 잘못된 토큰도 401 / INVALID_TOKEN 으로 거부되는 것을 확인하였다.

관련 코드: backend-fastapi/auth.py / current_user_id()
관련 테스트:
- T09: 변조된 JWT 검사
- T10: 만료된 JWT 검사
- 추가 검사: aud 클레임 불일치
증거:
- docs/evidence/07_invalid_token.png
- docs/evidence/08_expired_token.png
- docs/evidence/31_invalid_audience.png

4. 로그아웃, 토큰 만료, 새로고침은 각각 무엇을 바꾸는가? 이미 복사된 토큰은 어떻게 되는가?

[내 웹서비스는 JWT 를 localStorage 나 sessionStorage 에 저장하지 않고, frontend-react/src/api.js 의 JavaScript 메모리에 있는 session 객체에서 관리한다.

로그아웃하면 clearSession() 함수가 메모리에 저장된 JWT 와 사용자 정보를 삭제하고 React 화면을 로그아웃 상태로 변경한다. 그러나 서버에서 이미 발급한 JWT 자체를 즉시 폐기하는 것은 아니다.

토큰 만료는 JWT 의 exp 에 지정된 유효 시간이 지난 상태를 의미한다. 만료된 JWT 로 보호 API 를 요청하면 FastAPI 서버가 HTTP 401 / TOKEN_EXPIRED 를 반환한다. React 는 HTTP 401 응답을 받으면 메모리의 인증 정보를 초기화하여 재로그인이 필요한 상태로 변경한다.

새로고침하면 JavaScript 실행 환경이 다시 초기화되므로 메모리에 저장되어 있던 JWT 가 사라지고 로그인 상태가 해제된다. 다만 새로고침 자체가 서버에 발급된 JWT 를 무효화하지는 않는다.

이미 다른 곳에 복사해 둔 JWT 는 로그아웃이나 새로고침만으로 무효화되지 않는다. 현재 구현에는 별도의 토큰 폐기 목록이 없으므로, 해당 JWT 의 서명과 클레임이 올바르고 유효 시간이 남아 있다면 만료 전까지 보호 API 에 사용할 수 있다. 반면 exp 가 지난 토큰은 복사본이라도 서버에서 거부한다.

T10 에서 만료된 토큰의 HTTP 401 / TOKEN_EXPIRED 응답과 React 로그인 상태 초기화를 확인하였고,
T11 에서 로그아웃 후 토큰 없는 요청이 HTTP 401 / TOKEN_REQUIRED 로 처리되는 것을 확인하였다.
T16 에서는 새로고침 후 로그인 상태가 해제되고 공개 게시글 조회는 유지되는지 검사하였다.

관련 코드:
frontend-react/src/api.js / clearSession(), request()
frontend-react/src/main.jsx / App()
backend-fastapi/auth.py / current_user_id()
관련 테스트: T10, T11, T16
증거:
docs/evidence/08_expired_token.png
docs/evidence/12_logout.png
docs/evidence/25_T10_expired_ui.png
docs/evidence/26_T11_logout_api.png

5. 가장 어려웠던 오류 한 가지와 원인 확인·수정·재검증 과정을 설명하라.

가장 어려웠던 오류는 T14 테스트에서 존재하지 않는 게시글을 조회·수정·삭제하는 과정에 발생한 PowerShell 자료형 변환 오류였다.

처음 테스트 명령을 실행했을 때 System.Object[]형식의 값을 System.Int32 형식으로 변환할 수 없다는 오류가 발생하였다. 처음에는 FastAPI 서버의 게시글 ID 처리 문제라고 생각했지만, 오류 메시지를 확인한 결과 서버에서 반환한 HTTP 오류가 아니라 PowerShell 테스트 코드에서 배열 값을 정수로 처리하는 과정의 문제임을 알게 되었다.

문제 해결을 위해 테스트 명령에서 배열 형태의 값을 사용하지 않고, 존재하지 않는 게시글 ID 를 999999999 라는 단일 숫자로 지정하였다. 이후 수정된 명령으로 같은 테스트를 다시 실행하였다.

재검증 결과 정상적인 Alice JWT 로 존재하지 않는 게시글에 수정 요청을 보냈을 때 HTTP 404 / NOT_FOUND 가 반환되는 것을 확인하였다. 또한 JWT 없이 동일한 게시글에 수정 요청을 보냈을 때는 HTTP 401 / TOKEN_REQUIRED 가 반환되었다.

이 과정을 통해 오류가 발생했을 때 서버 구현 문제와 테스트 도구의 실행 오류를 구분하는 것이 중요하다는 점을 알게 되었다. 또한 AI 가 제안한 명령어를 그대로 사용하는 것보다 실제 오류 메시지와 HTTP 응답을 직접 확인하고 수정 후 재검증하는 과정이 필요하다는 것

현재 남은 문제와 다음에 개선할 점: [없으면 없음으로 표시]
현재 기본 JWT 인증, 게시글 CRUD, 작성자 권한 검사 및 SQLite 데이터 유지 기능은 구현하였다.

다만 현재 로그아웃은 브라우저 메모리의 JWT 를 삭제하는 방식이므로, 이미 복사된 JWT 를 서버에서 즉시 폐기하는 기능은 없다. 향후에는 토큰 폐기 목록이나 별도의 세션 관리 기능을 도입하여 로그아웃 시 서버에서도 토큰을 무효화할 수 있도록 개선하고 싶다.

또한 사용자가 인증 만료 시점을 더 쉽게 알 수 있도록 남은 로그인 시간을 표시하고, 모바일 환경에서도 사용하기 편리하도록 반응형 화면 구성을 개선할 계획이다.

제출 확인: [ ] 실제 코드와 설명 일치 [ ] 테스트 직접 수행 [ ] AI 활용 및 출처 기록 [ ] 비밀 키와 토큰 제거

GitHub 확인: [ ] 루트 README 작성 [ ] 보고서 PDF 와 증거 업로드 [ ] README 문서 링크 확인 [ ] 교수자 접근 가능 [ ] 저장소 URL 과 커밋 SHA 제출